

Reference Notes

Week 10: 2D Lists and Matrix Operations

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

How can we organize data in multiple dimensions, and why do some data types behave differently when we try to change them? This week, we'll explore how lists can contain other lists, creating powerful grid-like structures that help us solve complex problems like building game boards and managing tabular data.

Prerequisites

You should be comfortable with:

- Creating and modifying regular lists
- Understanding list mutability and aliasing
- Using list methods and indexing
- Drawing memory diagrams for lists

1.2 What You Already Know

You've mastered single lists - creating them, modifying them, and understanding how they behave in memory. You know that lists are mutable, which means we can change their contents. You've also learned about aliasing and how multiple variables can point to the same list object. These concepts are the foundation for understanding 2D lists.

1.3 What You'll Be Able to Do

By the end of this week, you'll be able to create game boards, work with grids of data, and understand exactly how nested lists work in memory. You'll build programs that can handle tic-tac-toe games, create pixel art, and process tabular data like spreadsheets. Most importantly, you'll understand the common pitfalls of creating 2D lists and how to avoid them.

2 VIDEO 1: Understanding 2D Lists

2.1 What Are 2D Lists?

A 2D list is simply a list that contains other lists as its elements. Think of it like a grid or table - you have rows and columns of data. In real life, this is like a classroom seating chart, a chess board, or even the pixels on your screen. Each position in the grid can be accessed using two indices: one for the row and one for the column.

2.2 Your First 2D List

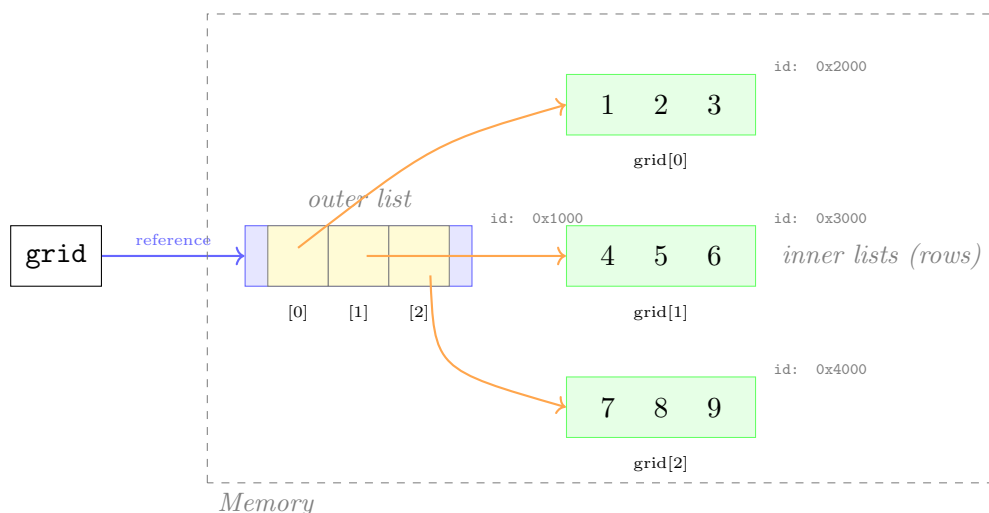
Let's create a simple 2D list representing a 3x3 grid, like a small tic-tac-toe board. We'll start with numbers to see how the structure works:

```
1 # Creating a 2D list manually
2 grid = [[1, 2, 3],
3         [4, 5, 6],
4         [7, 8, 9]]
5
6 # Accessing elements: grid[row][column]
7 print(grid[0][0]) # First row, first column: 1
8 print(grid[1][2]) # Second row, third column: 6
```

Notice how we use two sets of square brackets to access elements. The first index selects the row (which is itself a list), and the second index selects the element within that row.

2.3 Visualizing 2D Lists in Memory

Understanding how 2D lists are stored in memory is crucial for avoiding common mistakes. Let's visualize what our grid looks like:



This diagram reveals something important: a 2D list is actually a list of references to other lists. Each row is a separate list object in memory, and the main list just holds references to these row lists.

2.4 Modifying Elements in a 2D List

Since we understand that 2D lists are lists of lists, let's see how modification works. We can change individual elements or entire rows:

```

1 # Changing a single element
2 grid[1][1] = 0 # Change center to 0
3 print(grid)    # [[1, 2, 3], [4, 0, 6], [7, 8, 9]]
4
5 # Replacing an entire row
6 grid[0] = ['X', '0', 'X']
7 print(grid)    # [['X', '0', 'X'], [4, 0, 6], [7, 8, 9]]

```

When we modify a single element, we're reaching into a specific row and changing one value. When we replace an entire row, we're changing what the outer list references at that position.

Tip

Always remember: 2D lists use row-first indexing. Think "which row?" first, then "which column in that row?"

Quick Check

What will this print?

```

1 matrix = [['a', 'b'], ['c', 'd']]
2 print(matrix[1][0])

```

3 VIDEO 2: Creating Nested List Structures

3.1 Building 2D Lists the Right Way

When creating larger 2D lists, typing out every element manually isn't practical. We need programmatic ways to create them. Let's explore the right and wrong ways to do this.

3.2 The Dangerous Shortcut

Here's a tempting but incorrect way to create a 2D list that causes many bugs:

```

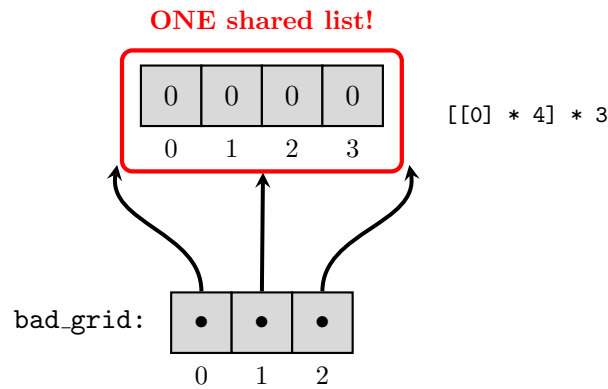
1 # WRONG WAY - Don't do this!
2 rows = 3
3 cols = 4
4 bad_grid = [[0] * cols] * rows
5
6 # Looks fine at first...
7 print(bad_grid) # [[0,0,0,0], [0,0,0,0], [0,0,0,0]]
8
9 # But watch what happens:
10 bad_grid[0][0] = 1
11 print(bad_grid) # [[1,0,0,0], [1,0,0,0], [1,0,0,0]]

```

Yikes! We changed one element but all the first elements in each row changed! This happens because the multiplication operator for lists creates references to the same list object, not separate copies.

3.3 Understanding the Reference Problem

Let's visualize why the wrong approach fails. This is one of the most important concepts to understand about 2D lists:



This diagram shows the critical issue: all rows are actually the same list object in memory. When you modify one, you're modifying the only row that exists, which appears three times!

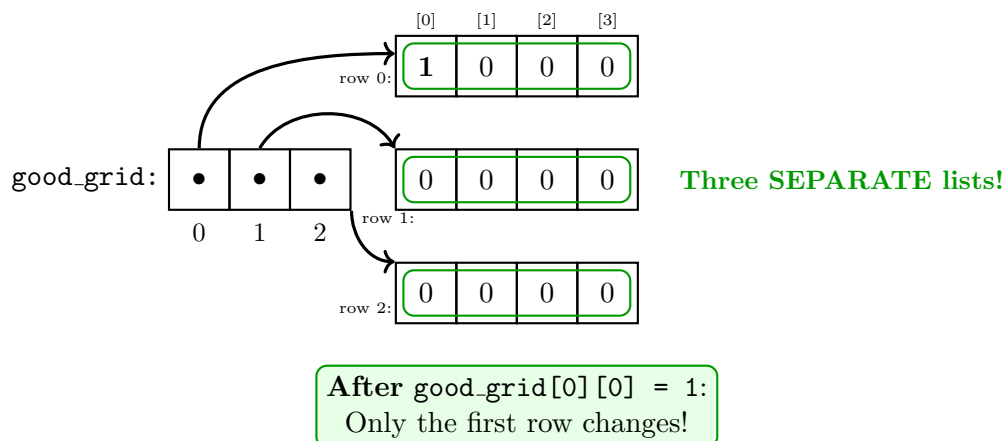
3.4 The Correct Approach

Here's how to properly create a 2D list where each row is independent:

```

1  # CORRECT WAY - Using list comprehension
2  rows = 3
3  cols = 4
4  good_grid = [[0 for j in range(cols)] for i in range(rows)]
5
6  # Now each row is independent
7  good_grid[0][0] = 1
8  print(good_grid)  # [[1,0,0,0], [0,0,0,0], [0,0,0,0]]
9
10 # Alternative: Using loops
11 grid2 = []
12 for i in range(rows):
13     row = [] # Create a NEW list for each row
14     for j in range(cols):
15         row.append(0)
16     grid2.append(row)
    
```

The following diagram illustrates why this method works. Instead of all rows pointing to the same list, each row points to a different list object.



The key insight is that we create a new list for each row. The list comprehension does this automatically, creating a fresh list object for each iteration of the outer loop.

⚠ Common Error

Common Mistake: Using `[[0]*n]*m` to create a 2D list. This creates references to the same row! Always use list comprehension or loops to ensure each row is a separate list.

👉 Quick Check

Will this create independent rows? Why or why not?

```
1 grid = [[0, 0, 0] for i in range(2)]
2 grid[0][0] = 5
```

4 VIDEO 3: Working with Matrix Operations

4.1 Traversing 2D Lists

To work with every element in a 2D list, we use nested loops. The outer loop goes through rows, and the inner loop goes through columns within each row. This pattern is fundamental to matrix operations.

4.2 Row-by-Row Processing

Let's start with the most common pattern - processing a 2D list row by row:

```
1 # Create a simple number grid
2 matrix = [[1, 2, 3],
3           [4, 5, 6],
4           [7, 8, 9]]
5
6 # Process each element
7 for row in range(len(matrix)):
8     for col in range(len(matrix[row])):
9         print(f"Position [{row}][{col}] = {matrix[row][col]}")
```

This nested loop pattern visits every element in order: first row left to right, then second row left to right, and so on. It's like reading a book - line by line, left to right.

4.3 Common Matrix Patterns

Working with 2D lists often involves recognizing and using common patterns. Let's explore some essential ones that you'll use repeatedly:

```
1 # Sum all elements
2 total = 0
3 for row in matrix:
4     for element in row:
5         total += element
6
7 # Find maximum in entire grid
8 max_val = matrix[0][0]
9 for row in matrix:
10     for element in row:
11         if element > max_val:
```

```

12         max_val = element
13
14 # Count specific values (e.g., zeros)
15 zero_count = 0
16 for row in matrix:
17     zero_count += row.count(0)

```

Notice how we can use both index-based loops (with range) and value-based loops (for element in row) depending on whether we need the position or just the value.

4.4 Building a Tic-Tac-Toe Board

Let's apply what we've learned to create a real game board. Tic-tac-toe is perfect for demonstrating 2D lists because it's a 3x3 grid where we need to check rows, columns, and diagonals:

```

1 # Initialize empty board
2 board = [[' ' for j in range(3)] for i in range(3)]
3
4 # Make some moves
5 board[1][1] = 'X' # Center
6 board[0][0] = 'O' # Top-left
7 board[2][2] = 'X' # Bottom-right
8
9 # Display the board nicely
10 for row in board:
11     print(' '.join(row))
12     if row != board[-1]: # Not last row
13         print('-+-')

```

This creates a visual representation of the board. The key is using 2D list indexing to place moves and nested loops to display the board.

💡 Rule

2D lists are perfect for grid-based data: game boards, images, spreadsheets, and anywhere you need rows and columns of information.

👉 Quick Check

How would you check if all elements in the first row of a 3x3 board are 'X'?

5 VIDEO 4: Summary

5.1 What We Learned This Week

This week, we discovered that 2D lists are simply lists containing other lists, giving us the power to work with grid-like data structures. We learned the critical difference between creating 2D lists correctly (where each row is independent) and incorrectly (where all rows reference the same list). We explored how to traverse these structures with nested loops and apply them to real problems like game boards.

The key insight is understanding that 2D lists are references to references - the outer list holds references to row lists, and each row list holds the actual data. This mental model helps us predict behavior and avoid the common aliasing trap when creating 2D lists.

★ Landmark Moment

You now understand multi-dimensional data structures! You can create game boards, work with grids, and handle table-like data - opening up a whole new class of programs you can write.

6 Quick Reference

📖 Quick Reference

Week 10 Essential Patterns:

Creating 2D Lists:

- Pattern: `[[val for j in range(cols)] for i in range(rows)]`
- Common use: Game boards, grids, matrices
- Common mistake: Using `[[0]*cols]*rows` creates aliased rows

Accessing Elements:

- Pattern: `grid[row][col]`
- Memory: First index selects row list, second selects element
- Debug tip: Print row and column indices while traversing

Traversing 2D Lists:

- Pattern: Nested loops - outer for rows, inner for columns
- Common use: Process all elements, search, calculate totals
- Debug tip: Use descriptive names like 'row' and 'col', not just 'i' and 'j'

6.1 Reflection Questions

1. What was the most challenging part about understanding 2D lists versus regular lists?
2. Can you explain to a friend why `[[0]*3]*3` doesn't work for creating independent rows?
3. When debugging a 2D list problem, what's your strategy for tracking down errors?
4. How has understanding memory diagrams helped you work with nested structures?
5. What real-world problems could you solve now with 2D lists that you couldn't before?

Next week: We'll explore dictionaries - a completely different way to organize data using keys and values instead of positions!